SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	ITA05	**Course Title:**	COMPUTER VISION
CO Assessed:	CO3 – Precision (BL4)	**Assessment:**	Assessment Tool 1 – Experiments
Weightage:	40% of CIA	**Total Marks:**	50 (5 Questions × 10 Mark)
CO1 Statement:	*Analyze and extract image features using detection and matching techniques for effective image representation and comparison.(BL4).*		
Student Name:	P.Moshiya	**Reg. No.:**	192572142
Section / Batch:	2025	**Date:**	

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- This test contains 5 questions, each carrying 10 marks. Total: 50 Marks.
- No negative marking. Unanswered questions carry zero marks.
- No calculators or electronic devices permitted.
- Duration: 60 minutes.

Section / Topic	Focus	Q Nos	Marks
Types of Image Processing	Point, Neighborhood, Geometric Processing, Applications	1	10
Image Representation	Grayscale, Binary, Color Representation	2	10
Hough Transform	Line Detection, Circle Detection, Parameter Space	3	10
Scale Invariant Feature Matching (SIFT)	Keypoint Detection, Feature Matching, Scale & Rotation Invariance	4	10
Principal Component Analysis (PCA)	Dimensionality Reduction, Feature Selection, Data Representation	5	10
	GRAND TOTAL:		50 Marks

Annexure A – Experiments

1.Feature Descriptor Comparison (SIFT vs Harris)

Design and perform an experiment to compare Harris Corner Detection and SIFT feature extraction on the same set of images. Explain the theoretical principles behind corner detection and feature descriptors. Implement both techniques using suitable software/tools, document the detected features systematically, and analyze their performance in terms of feature quality, robustness, repeatability, and suitability for image matching applications.

2.Image Enhancement for Feature Detection

Conduct an experiment to evaluate the effect of image enhancement techniques such as histogram equalization and contrast adjustment on feature detection performance. Explain the importance of image enhancement in computer vision. Apply enhancement methods followed by feature detection using appropriate tools, document the intermediate and final outputs clearly, and analyze how enhancement influences feature visibility, detection accuracy, and reliability.

3.Multi-Scale Feature Detection Analysis

Design and implement an experiment to study feature detection at different image scales. Explain the concept of image scaling and scale-space representation in feature extraction. Apply feature detection methods on images of varying resolutions using suitable software/tools, document the detected features systematically, and analyze the effect of scale variations on feature stability, accuracy, and computational performance.

4.Comparative Analysis of Image Filtering Techniques

Conduct an experiment to compare different image filtering techniques, such as Mean, Gaussian, and Median filtering, for preprocessing. Explain the theoretical concepts and objectives of each filtering method. Implement the filters using appropriate tools/software, document the processed images and observations clearly, and analyze their effectiveness in noise reduction while preserving important image features for subsequent feature detection.

5.Complete Object Detection Workflow

Design and perform an experiment to develop an object detection workflow incorporating image preprocessing, edge detection, feature extraction, and shape detection. Explain the purpose and interaction of each processing stage in the workflow. Implement the complete pipeline using suitable software/tools, document the outputs at every stage systematically, and analyze the effectiveness of the integrated approach in achieving accurate and reliable object detection under different image conditions.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts	
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation	
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools	
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results	
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation	

Faculty In-charge
Signature: _____
Date: _____

1. Feature Descriptor Comparison – SIFT vs Harris

Aim

To compare **Harris Corner Detection** and **SIFT feature extraction** on the same image and analyze their feature quality, robustness, repeatability and suitability for image matching.

Theory

Harris Corner Detection

Harris detects corners by analyzing the change in image intensity when a small window is shifted in different directions. A corner produces a large intensity change in both directions.

Harris mainly detects **corner points**. It does not itself provide a powerful descriptor for matching.

SIFT

SIFT stands for **Scale-Invariant Feature Transform**. It detects keypoints and creates descriptors that are robust to:

- Scale changes
- Rotation
- Moderate illumination changes
- Image transformations

Therefore, SIFT is more suitable for **feature matching**.

Algorithm

1. Read the input image.
2. Convert the image to grayscale.
3. Apply Harris Corner Detection.
4. Detect SIFT keypoints and descriptors.
5. Draw the detected features.
6. Display both results.
7. Compare the number and quality of detected features.

Program

```
import cv2

# Read image
img = cv2.imread("input.jpg")

# Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Harris Corner Detection
gray_float = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```
gray_float = cv2.GaussianBlur(gray_float, (3, 3), 0)

harris = cv2.cornerHarris(gray_float, 2, 3, 0.04)

harris_img = img.copy()
harris_img[harris > 0.01 * harris.max()] = [0, 0, 255]

# SIFT Feature Detection
sift = cv2.SIFT_create()
keypoints, descriptors = sift.detectAndCompute(gray, None)

sift_img = cv2.drawKeypoints(
    img,
    keypoints,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

print("Number of Harris corners:",
      (harris > 0.01 * harris.max()).sum())

print("Number of SIFT keypoints:", len(keypoints))

cv2.imshow("Harris Corners", harris_img)
cv2.imshow("SIFT Features", sift_img)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input

An image such as:

input.jpg

Use an image containing objects, corners, edges and textures.

Output

Two windows are obtained:

1. **Harris Corners** – red points indicate detected corners.
2. **SIFT Features** – keypoints with orientation and scale information are displayed.

Analysis

Feature	Harris	SIFT
Detects	Corners	Keypoints
Scale invariant	No	Yes
Rotation invariant	Limited	Yes
Descriptor	No	Yes
Matching suitability	Moderate	High
Computational cost	Low	Higher

Result

Thus, **Harris Corner Detection** successfully detects corner points, while **SIFT** detects distinctive keypoints and generates descriptors. SIFT provides better robustness and is more suitable for image matching applications.

2. Image Enhancement for Feature Detection

Aim

To study the effect of **histogram equalization and contrast enhancement** on feature detection performance.

Theory

Image enhancement improves image quality by increasing the visibility of important structures such as edges, corners and textures.

Histogram Equalization

Histogram equalization redistributes intensity values to improve overall contrast.

Contrast Adjustment

Contrast stretching expands the intensity range of an image so that dark and bright regions become more distinguishable.

Feature detectors can detect features more reliably when important image structures are clearly visible.

Algorithm

1. Read the input image.
2. Convert it to grayscale.
3. Apply original image feature detection.
4. Apply histogram equalization.
5. Apply contrast adjustment.
6. Perform feature detection on enhanced images.
7. Compare the number of detected features.

Program

```
import cv2

# Read image
img = cv2.imread("input.jpg")

# Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Histogram Equalization
equalized = cv2.equalizeHist(gray)

# Contrast Adjustment
contrast = cv2.convertScaleAbs(gray, alpha=1.5, beta=20)

# SIFT detector
sift = cv2.SIFT_create()

# Original image features
kp1, des1 = sift.detectAndCompute(gray, None)

# Equalized image features
kp2, des2 = sift.detectAndCompute(equalized, None)

# Contrast enhanced image features
kp3, des3 = sift.detectAndCompute(contrast, None)
```

```
# Draw features
out1 = cv2.drawKeypoints(gray, kp1, None)
out2 = cv2.drawKeypoints(equalized, kp2, None)
out3 = cv2.drawKeypoints(contrast, kp3, None)

print("Original features:", len(kp1))
print("Histogram Equalized features:", len(kp2))
print("Contrast Enhanced features:", len(kp3))

cv2.imshow("Original", gray)
cv2.imshow("Histogram Equalization", equalized)
cv2.imshow("Contrast Adjustment", contrast)

cv2.imshow("Original Features", out1)
cv2.imshow("Equalized Features", out2)
cv2.imshow("Contrast Features", out3)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input

input.jpg

Preferably use an image with low contrast or uneven illumination.

Output

The following outputs are produced:

- Original grayscale image
- Histogram equalized image
- Contrast enhanced image
- Features detected on the original image
- Features detected after histogram equalization
- Features detected after contrast adjustment

Analysis

Method	Effect
Original	Normal feature visibility
Histogram Equalization	Improves global contrast
Contrast Adjustment	Makes details more visible
Enhanced Image	May produce clearer/more detectable features

Enhancement can improve feature visibility, particularly in low-contrast images. However, excessive enhancement can also introduce unwanted details or noise.

Result

Thus, image enhancement improves the visibility of important image structures and can improve feature detection reliability. **Histogram equalization and contrast adjustment are useful preprocessing techniques for feature detection.**

3. Multi-Scale Feature Detection Analysis

Aim

To study feature detection at different image scales and analyze the effect of resolution on feature stability, accuracy and computational performance.

Theory

Image Scaling

Image scaling changes the dimensions or resolution of an image.

For example:

Original → 100%

Medium → 75%

Small → 50%

Scale Space

Scale-space representation analyzes an image at different levels of resolution. Feature detectors such as SIFT are designed to identify important features across different scales.

A good scale-invariant detector should detect similar important features even when the image size changes.

Algorithm

1. Read the original image.
2. Create images at 100%, 75%, 50% and 25% scales.

3. Apply SIFT feature detection to each image.
4. Count the detected keypoints.
5. Measure processing time.
6. Compare the results.

Program

```
import cv2
import time

# Read image
img = cv2.imread("input.jpg")
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# SIFT detector
sift = cv2.SIFT_create()

scales = [1.0, 0.75, 0.50, 0.25]

for scale in scales:

    # Resize image
    resized = cv2.resize(
        gray,
        None,
        fx=scale,
        fy=scale
    )

    # Start timer
    start = time.time()

    # Detect features
    keypoints, descriptors = sift.detectAndCompute(
        resized, None
```

```
    )

    end = time.time()

    # Draw features
    result = cv2.drawKeypoints(
        resized,
        keypoints,
        None,
        flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
    )

    print("Scale:", scale)
    print("Keypoints:", len(keypoints))
    print("Time:", round(end - start, 4), "seconds")

    cv2.imshow(
        "Scale " + str(scale),
        result
    )

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input

input.jpg

Output

Feature-detected images at:

100%

75%

50%

25%

The terminal displays:

Scale: 1.0

Keypoints: ...

Scale: 0.75
Keypoints: ...

Scale: 0.5
Keypoints: ...

Scale: 0.25
Keypoints: ...

Analysis

Scale	Feature Stability	Processing Time
100%	High	Higher
75%	High	Medium
50%	Moderate	Lower
25%	Lower	Very low

As image resolution decreases, some small features disappear. At the same time, computation becomes faster because fewer pixels are processed.

Result

Thus, feature detection at multiple scales shows that **higher resolutions preserve more details**, while **lower resolutions reduce computational cost**. SIFT provides good robustness to scale changes.

4. Comparative Analysis of Image Filtering Techniques

Aim

To compare **Mean, Gaussian and Median filters** for noise reduction and determine their effectiveness in preserving important image features.

Theory

Mean Filter

The mean filter replaces each pixel with the average of its neighboring pixels.
It reduces noise but can blur edges.

Gaussian Filter

The Gaussian filter gives greater weight to pixels near the center.

It provides smooth noise reduction while producing less distortion than a simple mean filter.

Median Filter

The median filter replaces each pixel with the median of neighboring pixel values.

It is particularly effective against **salt-and-pepper noise** and preserves edges better.

Algorithm

1. Read the input image.
2. Convert it to grayscale.
3. Add noise to the image.
4. Apply mean filtering.
5. Apply Gaussian filtering.
6. Apply median filtering.
7. Display and compare the results.

Program

```
import cv2
import numpy as np

# Read image
img = cv2.imread("input.jpg")

# Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Add salt and pepper noise
noisy = gray.copy()

noise = np.random.choice(
    [0, 255],
    size=gray.shape,
    p=[0.95, 0.05]
)
```

```
noisy = np.where(noise == 255, 255, noisy).astype(np.uint8)

# Mean filter
mean = cv2.blur(noisy, (5, 5))

# Gaussian filter
gaussian = cv2.GaussianBlur(
    noisy,
    (5, 5),
    0
)

# Median filter
median = cv2.medianBlur(
    noisy,
    5
)

# Display
cv2.imshow("Original", gray)
cv2.imshow("Noisy Image", noisy)
cv2.imshow("Mean Filter", mean)
cv2.imshow("Gaussian Filter", gaussian)
cv2.imshow("Median Filter", median)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input

input.jpg

Output

The program displays:

1. Original image
2. Noisy image

3. Mean filtered image
4. Gaussian filtered image
5. Median filtered image

Analysis

Filter	Noise Reduction	Edge Preservation
Mean	Good	Low
Gaussian	Good	Moderate
Median	Very good for salt-and-pepper	High

Observation

- Mean filtering smooths the image but may blur edges.
- Gaussian filtering produces smooth and natural-looking results.
- Median filtering removes impulse noise while preserving edges.

Result

Thus, **Median filtering is most effective for salt-and-pepper noise**, while Gaussian filtering provides good general-purpose smoothing. Mean filtering is simple but can cause greater edge blurring.

5. Complete Object Detection Workflow

Aim

To develop a complete object detection workflow using **image preprocessing, edge detection, feature extraction and shape detection**.

Theory

A complete object detection pipeline can contain the following stages:

Input Image

↓

Preprocessing

↓

Edge Detection

↓

Feature Extraction

↓

Shape Detection

↓

Object Detection

1. Preprocessing

Improves image quality and reduces noise.

2. Edge Detection

Detects boundaries of objects. Canny edge detection is commonly used.

3. Feature Extraction

Extracts important points or structures from the image.

4. Shape Detection

Detects geometric shapes such as circles and contours.

5. Object Detection

Combines the information to identify objects.

Algorithm

1. Read the image.
2. Convert it to grayscale.
3. Apply Gaussian filtering.
4. Apply Canny edge detection.
5. Detect contours.
6. Find the contours' bounding boxes.
7. Extract SIFT features.
8. Draw detected objects and features.
9. Display all intermediate outputs.

Program

```
import cv2

# Read image
img = cv2.imread("input.jpg")

# Convert to grayscale
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Preprocessing
blur = cv2.GaussianBlur(gray, (5, 5), 0)
```

```
# Edge detection
edges = cv2.Canny(blur, 50, 150)

# Shape detection using contours
contours, hierarchy = cv2.findContours(
    edges,
    cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE
)

object_img = img.copy()

for contour in contours:

    # Remove very small objects
    area = cv2.contourArea(contour)

    if area > 500:

        x, y, w, h = cv2.boundingRect(contour)

        cv2.rectangle(
            object_img,
            (x, y),
            (x + w, y + h),
            (0, 255, 0),
            2
        )

        cv2.putText(
            object_img,
            "Object",
```

```
            (x, y - 10),
            cv2.FONT_HERSHEY_SIMPLEX,
            0.6,
            (0, 255, 0),
            2
        )

# Feature extraction using SIFT
sift = cv2.SIFT_create()

keypoints, descriptors = sift.detectAndCompute(
    gray,
    None
)

features = cv2.drawKeypoints(
    img,
    keypoints,
    None,
    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

print("Number of detected features:", len(keypoints))
print("Number of detected objects:",
      sum(cv2.contourArea(c) > 500 for c in contours))

# Display stages
cv2.imshow("Original Image", img)
cv2.imshow("Preprocessed Image", blur)
cv2.imshow("Canny Edges", edges)
cv2.imshow("SIFT Features", features)
cv2.imshow("Detected Objects", object_img)
```

```
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input

input.jpg

Use an image containing clearly visible objects.

Output

The program produces:

1. Original Image

The input image.

2. Preprocessed Image

Noise is reduced using Gaussian filtering.

3. Canny Edges

Object boundaries are detected.

4. SIFT Features

Important keypoints are detected.

5. Detected Objects

Objects are enclosed using rectangular bounding boxes.

Analysis

Stage	Purpose	Output
Preprocessing	Remove noise	Smooth image
Edge Detection	Find boundaries	Edge image
Feature Extraction	Find important points	Keypoints
Shape Detection	Identify object boundaries	Contours
Object Detection	Locate objects	Bounding boxes

The performance depends on image quality, lighting, background, object size and noise level.

Result

Thus, the complete workflow successfully combines **preprocessing, Canny edge detection, SIFT feature extraction and contour-based shape detection** to identify objects. The integrated approach provides a systematic method for object detection under different image conditions.